

Lezione 12 - Multithreading Avanzato

RIDUZIONE NEI CICLI

◆ Problema

👉 un pattern molto comune nei cicli `for` è:

- accumulare un valore in una variabile condivisa

👉 esempi:

- somma degli elementi di un array
- verificare una condizione su tutti gli elementi

◆ Esempio tipico

👉 codice classico:



Codice:

```
sum += v[i];
```

👉 questo avviene:

- ad ogni iterazione del ciclo

◆ Problema con OpenMP

👉 se parallelizziamo il ciclo:

```
#pragma omp parallel for
```

👉 allora:

- più thread aggiornano `sum`

✳️ problema: *race condition*

◆ Perché?

👉 `sum` è:

- una variabile condivisa

👉 quindi:

- più thread scrivono contemporaneamente

✳️ risultato non deterministico

◆ Soluzione manuale

👉 possiamo risolvere così:

- ogni thread usa una variabile *locale*
- calcola una *somma parziale*
- alla fine si combinano i risultati

◆ Idea

👉 schema:

- `partial_sum` → locale al thread
- `sum` → condivisa

👉 alla fine:

- sommiamo le somme parziali

◆ Problema della soluzione manuale

👉 richiede:

- gestione esplicita delle variabili
- uso di *sezioni critiche*

✳️ soluzione corretta ma scomoda

📌 REDUCTION IN OPENMP

◆ Idea

👉 OpenMP fornisce un modo automatico per gestire le riduzioni:

```
#pragma omp for reduction(op : var)
```

👉 dove:

- `op` → operazione (`*, -, &&, ||, ...`)
- `var` → variabile su cui effettuare la riduzione

◆ Funzionamento

👉 OpenMP:

- crea automaticamente una copia *locale* di `var` per ogni thread
- ogni thread calcola il proprio risultato
- alla fine combina i risultati

✳️ senza *race condition*

◆ Esempio base



Codice:

```
#pragma omp parallel for reduction(+: sum)
for (int i = 0; i < n; i++)
{
    sum += v[i];
}
```

◆ Spiegazione

👉 per ogni thread:

- viene creata una copia *locale* di `sum`

👉 durante il ciclo:

- ogni thread aggiorna la propria copia

👉 alla fine:

- OpenMP combina tutte le copie
- usando l'operazione `+`

★ otteniamo il risultato corretto

◆ Esempio completo

◆ Versione manuale (corretta ma scomoda)



Codice:

```
#pragma omp parallel
{
    int partial = 0;

    #pragma omp for nowait
    for (int i = 0; i < n; i++)
    {
        partial += v[i];
    }

    #pragma omp critical
    {
        sum += partial;
    }
}
```

👉 ogni thread:

- usa una variabile *locale* (partial)
- alla fine aggiorna sum in una *sezione critica*

✳️ corretto ma meno efficiente

◆ Versione con reduction (CORRETTA)



Codice:

```
#pragma omp parallel for reduction(+: sum)
for (int i = 0; i < n; i++)
{
    sum += v[i];
}
```

👉 OpenMP:

- crea copie *locali* di `sum`
- combina automaticamente i risultati

✳ soluzione migliore e più efficiente

◆ Versione sbagliata (DA NON FARE)



Codice:

```
#pragma omp parallel for
for (int i = 0; i < n; i++)
{
    sum += v[i];
}
```

👉 problema:

- `sum` è condivisa
- più thread la modificano

✳ *race condition* → risultato errato

◆ Idea importante

👉 reduction :

- evita sezioni critiche
- migliora le prestazioni
- semplifica il codice

✳ è il modo corretto per accumulare risultati

LIMITI DELLA REDUCTION

◆ Idea

👉 la direttiva `reduction` funziona molto bene, ma ha alcune limitazioni

◆ Vincolo principale

👉 nel corpo del ciclo, la variabile deve essere usata in forme specifiche

◆ Forme supportate

👉 OpenMP supporta solo queste operazioni su `var` :

- `var = var op expr`
- `var = expr op var`
- `var op= expr`

👉 esempi validi:

```
sum = sum + v[i];  
sum += v[i];
```

◆ Operazioni semplici

👉 sono supportate anche:

```
sum++;  
sum--;  
++sum;  
--sum;
```

👉 ma solo per:

- operatori + e -

◆ Limitazione importante

👉 l'espressione `expr` :

- non deve dipendere da `var`

👉 quindi:

✳️ operazioni complesse non sono supportate

◆ Conseguenza

👉 `reduction` funziona bene per:

- somme
- prodotti
- operazioni logiche

👉 ma NON per:

- logiche complesse
- dipendenze tra iterazioni

✳️ in questi casi serve una gestione manuale

◆ Idea importante

👉 `reduction` :

- è potente
- ma non universale

✳️ va usata quando il pattern è semplice

CICLI INNESTATI

◆ Problema

👉 consideriamo due cicli annidati:

Codice:

```
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        f(i, j);
    }
}
```

👉 vogliamo parallelizzare questo codice

◆ Soluzione standard

👉 in generale:

✳ si parallelizza il ciclo più esterno

◆ Esempio

Codice:

```
#pragma omp parallel for
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        f(i, j);
    }
}
```

◆ Comportamento

👉 le iterazioni del ciclo esterno (i):

- vengono divise tra i thread

👉 per ogni valore di i :

- il ciclo interno (j) viene eseguito *sequenzialmente*

◆ Distribuzione del lavoro

👉 esempio con 4 thread:

- thread 0 $\rightarrow i = 0$
- thread 1 $\rightarrow i = 1$
- thread 2 $\rightarrow i = 2$
- thread 3 $\rightarrow i = 3$

👉 per ogni i :

- vengono eseguite tutte le iterazioni di j

◆ Idea importante

👉 parallelizzare il ciclo esterno:

- evita sincronizzazioni inutili
- mantiene buona efficienza

✳ è la scelta più naturale

◆ Caso particolare

👉 se il ciclo esterno è molto piccolo:

- pochi valori di i
- molti valori di j

👉 allora:

✳️ il parallelismo è limitato

◆ Possibile alternativa

👉 possiamo parallelizzare il ciclo interno:



Codice:

```
for (int i = 0; i < n; i++)
{
    #pragma omp parallel for
    for (int j = 0; j < m; j++)
    {
        f(i, j);
    }
}
```

◆ Problema

👉 per ogni iterazione di i :

- viene creata una sezione parallela
- i thread devono sincronizzarsi

✳️ overhead elevato

◆ Conseguenza

👉 il ciclo esterno resta:

- *sequenziale*

👉 mentre quello interno:

- è *parallelo*

✳️ inefficiente nella maggior parte dei casi

◆ Idea importante

👉 nei cicli innestati:

- scegliere quale ciclo parallelizzare è fondamentale

✳️ di solito: ciclo più esterno

📌 CICLI INNESTATI — COSA NON FARE

◆ Problema

👉 alcune soluzioni sembrano corrette, ma in realtà non funzionano

◆ Errore 1: parallelizzare tutti i cicli

💻 **Codice:**

```
#pragma omp parallel for
for (int i = 0; i < n; i++)
{

#pragma omp parallel for
    for (int j = 0; j < m; j++)
    {
        f(i, j);
    }
}
```

◆ Perché è sbagliato?

👉 quando entriamo nel secondo `parallel for` :

- c'è già un solo thread attivo per quella iterazione
- non può dividere il lavoro con altri thread

✳️ non otteniamo parallelismo reale

◆ Errore 2: usare più `pragma omp for` inutili

👉 aggiungere più `#pragma omp for` :

- non migliora il parallelismo
- è equivalente a usarlo solo sul ciclo esterno

✳️ inutile

◆ Errore 3: ciclo `for` dentro `parallel` (sbagliato)



Codice:

```
#pragma omp parallel
for (int i = 0; i < n; i++)
{

#pragma omp for
    for (int j = 0; j < m; j++)
    {
        f(i, j);
    }
}
```

◆ Perché è sbagliato?

👉 il ciclo esterno viene eseguito da tutti i thread:

- ogni thread esegue tutte le iterazioni di i

👉 il ciclo interno viene diviso

✳️ il lavoro viene duplicato (Errore Grave)

◆ Conseguenza

👉 invece di migliorare:

- peggioriamo le prestazioni
- introduciamo errori logici

◆ Idea importante

👉 nei cicli innestati:

- non basta aggiungere pragma
- bisogna capire il modello di esecuzione

✳️ altrimenti si creano soluzioni sbagliate

📌 CICLI INNESTATI — SOLUZIONE CORRETTA

◆ Idea

👉 quando entrambi i cicli sono importanti:

✳️ possiamo “collassare” i cicli in uno solo

◆ Concetto

👉 invece di:

```
for (int i = 0; i < n; i++)  
{  
    for (int j = 0; j < m; j++)  
    {  
        f(i, j);  
    }  
}
```

👉 pensiamo a:

- un unico ciclo di lunghezza $n \times m$

◆ Soluzione manuale



Codice:

```
#pragma omp parallel for  
for (int ij = 0; ij < n*m; ij++)  
{  
    int i = ij / m;  
    int j = ij % m;  
    f(i, j);  
}
```

◆ Spiegazione

👉 trasformiamo:

- due indici $\rightarrow$ uno solo

👉 usando:

- divisione (/)
- modulo (%)

✳️ otteniamo tutte le coppie (i, j)

◆ Soluzione OpenMP

👉 OpenMP permette di farlo automaticamente:



Codice:

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        f(i, j);
    }
}
```

◆ Significato

👉 collapse(2) :

- unisce i 2 cicli più esterni
- crea un unico spazio di iterazioni

✳️ migliore parallelismo e bilanciamento

◆ Vantaggio

👉 con collapse :

- più lavoro da distribuire
- migliore bilanciamento

◆ Quando usarlo

👉 utile quando:

- il ciclo esterno è piccolo

- il ciclo interno è grande

✳️ aumenta il parallelismo

◆ Idea importante

👉 nei cicli innestati:

- collapse è la soluzione corretta

✳️ evita errori e migliora le prestazioni

📌 TASK PARALLELISM

◆ Idea

👉 finora abbiamo parallelizzato:

- cicli (`for`)

👉 ma a volte abbiamo:

- operazioni diverse
- indipendenti tra loro

✳️ possiamo usare i *task*

◆ Concetto

👉 invece di dividere iterazioni:

- definiamo *compiti* (task)

👉 i thread:

- prendono ed eseguono i task disponibili

✳️ modello più flessibile e dinamico

◆ Struttura

👉 tipicamente:



Codice:

```
#pragma omp parallel
#pragma omp single
{
    #pragma omp task
        f(2);

    #pragma omp task
        g(3);

    #pragma omp task
        f(12);
}
```

◆ Spiegazione

👉 parallel :

- crea i thread

👉 single :

- un solo thread crea i task

👉 task :

- definisce un lavoro indipendente
- verrà eseguito da uno dei thread

◆ Esecuzione

👉 i task:

- non vengono eseguiti in ordine
- vengono distribuiti ai thread disponibili

✳️ esecuzione *dinamica*

◆ Esempio reale



Codice:

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

void f(int x)
{
    int y = 0;
    for (int i = 0; i < 1<<x; i++)
    {
        y++;
    }
    printf("Il thread %d sta eseguendo f(%d) con risultato %d\n", omp_get_thread_num(), x, y);
}

int main(int argc, char *argv[])
{
    #pragma omp parallel
    #pragma omp single
    {
        #pragma omp task
        f(3);
        #pragma omp task
        f(13);
        #pragma omp task
        f(20);
        #pragma omp task
        f(11);
        #pragma omp task
        f(18);
        #pragma omp task
        f(16);
    }
    return 0;
}
```

◆ Spiegazione del codice

👉 ogni chiamata a $f(x)$:

- è un *task*

👉 i thread:

- eseguono i task disponibili
- possono lavorare su task diversi

✳️ bilanciamento automatico

◆ Vantaggi

👉 i task:

- sono flessibili
- permettono parallelismo irregolare
- funzionano bene con carichi diversi

◆ Idea importante

👉 quando il lavoro non è:

- regolare
- facilmente dividibile in cicli

✳️ usare i *task*